SIMATS ENGINEERING

ASSESSMENT TOOL II – REVERSE ENGINEERING TASK (MEDIUM)

Code of Conduct:

I **Delhi Ganesh P (192521434)** (Name / Reg. No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Delhi Ganesh P

Task 1 – FootPrintX.exe

1. Which functions or modules are responsible for gathering target information?

The responsible components would typically include DNS-resolution functions, IP/network enumeration routines, WHOIS or registration-query modules, and socket/network-information APIs. During reverse engineering, an analyst would look for imports and call sequences related to functions such as getaddrinfo(), gethostbyname(), DNS query APIs, socket(), connect(), and HTTP/HTTPS request routines. If the binary contains custom routines, their role can be confirmed by tracing calls from the main execution flow to these network and parsing functions.

2. What footprinting techniques are implemented in the executable?

Based on the described behavior, the executable may implement passive and active footprinting techniques such as domain-to-IP resolution, DNS record enumeration, network information collection, WHOIS/registration lookups, and possibly hostname or subdomain discovery. Passive techniques obtain information from public sources, while active techniques directly query target infrastructure.

3. How can the information collection process be disabled or modified?

The safest approach is to disable the network-collection component through configuration, remove or block its network-access capability, or isolate the executable in a controlled analysis environment. For authorized research, an analyst can patch or instrument the relevant routine so that it returns an empty result or stops before making external requests. Network firewall rules, DNS filtering, and application allowlisting can also prevent unwanted collection without modifying the binary.

Task 2 – DNSScan.exe

1. Which function is responsible for initiating DNS requests?

The initiating routine would normally be a DNS-resolution or DNS-query function. In a Windows executable, an analyst would check imports and cross-references for APIs such as DnsQuery(), GetAddrInfoW(), getaddrinfo(), or related DNS resolver functions. The exact function can only be named confidently after examining the actual binary.

2. Does the application attempt an AXFR (Zone Transfer) request? Justify your answer.

The application should be considered an AXFR client if reverse engineering shows that it constructs a DNS query with the AXFR query type and sends it to an authoritative DNS server, usually over TCP because DNS zone transfers commonly use TCP. Useful indicators include the string or constant associated with AXFR, DNS message-building code containing the AXFR record type, TCP connections to authoritative name servers, and parsing of multiple DNS records in the returned response. Without the binary, an actual AXFR attempt cannot be confirmed.

3. What information can an attacker obtain using the extracted DNS records?

A successful zone transfer can expose hostnames and IP addresses, mail-exchange (MX) servers, name servers, subdomains, service-related records, aliases, TXT records, and other DNS configuration details. This information can help attackers map an organization's infrastructure, identify externally exposed services, discover naming patterns, and select targets for further attacks.

Task 3 – IntelGather.exe

1. What types of information are collected by the application?

The described application may collect publicly available employee names, job titles, email addresses, telephone numbers, organization names, domain names, social-media references, technology-stack information, public documents, and other organizational metadata. It may also collect information about software platforms, web technologies, cloud services, and publicly visible infrastructure.

2. Which external websites or services are accessed during execution?

An analyst would identify external services by inspecting URL strings, imported networking libraries, HTTP headers, TLS connections, DNS requests, and captured traffic. Possible categories include search engines, public company websites, professional/social networking sites, code repositories, certificate-transparency services, DNS/WHOIS services, and public technology-identification APIs. The exact websites cannot be confirmed without the executable or network trace.

3. How can the collected intelligence be misused by attackers?

Attackers can use the information to identify employees for phishing and social engineering, construct believable pretexts, discover technologies and exposed services, identify high-value personnel, map an organization's external attack surface, and prioritize vulnerable systems. Combining several public data sources can make targeted attacks considerably more convincing.

Task 4 – MailCraft.exe

1. Which functions are responsible for generating phishing emails?

The relevant routines would include email-template generation, string-formatting, recipient-selection, and message-sending functions. During analysis, an analyst would look for SMTP or mail-library APIs, HTTP-based mail-service requests, MIME/message construction routines, and functions that combine templates with recipient-specific data.

2. What information is used to personalize the messages?

Personalization may use recipient names, job titles, department names, organization names, email addresses, account or service names, and other information obtained from files, databases, configuration data, or public sources. Templates may also insert dates, links, sender identities, or organization branding to make the messages appear legitimate.

3. How can the phishing functionality be prevented or removed?

Organizations should remove or quarantine the malicious executable, block its network destinations, restrict unauthorized SMTP or mail-service access, and use endpoint detection and response tools to identify related processes and persistence. Email security controls such as anti-phishing filters, DMARC/SPF/DKIM enforcement, URL scanning, attachment sandboxing, and user awareness training can reduce the impact. In a controlled lab, the email-generation routine can be disabled or instrumented so that no messages are actually sent.

Task 5 – PortalClone.exe

1. How does the application capture usernames and passwords?

A credential-stealing clone commonly presents a fake login form and captures the values entered into its username and password fields. In reverse engineering, analysts would inspect form-handling code, input-event handlers, HTTP request construction, and local storage or file-writing routines. The exact mechanism depends on the binary and cannot be confirmed without examining it.

2. Where are the captured credentials stored or transmitted?

Possible destinations include local text or database files, temporary files, Windows registry locations, or a remote attacker-controlled server through HTTP/HTTPS, sockets, or another communication channel. Static analysis can reveal file paths, URLs, IP addresses, and network libraries; dynamic analysis can confirm the destination by observing file-system and network activity in an isolated sandbox.

3. What security risks arise from the use of such applications?

The primary risks are credential theft, account takeover, identity compromise, unauthorized access to corporate systems, data breaches, financial loss, and follow-on attacks using stolen credentials. Reused passwords can expose multiple accounts. Such tools can also undermine trust in legitimate login portals and enable large-scale phishing campaigns.

Conclusion

Reverse engineering helps analysts identify how suspicious binaries collect information, communicate with external services, generate phishing content, or capture credentials. Static analysis reveals imports, strings, control flow, and embedded configuration, while dynamic analysis confirms actual behavior through controlled execution and network monitoring. Because the executable samples were not supplied, exact function names, URLs, storage locations, and AXFR behavior must be verified against the corresponding binaries.